



Trial final exam with solutions - 2024

Introduction to python (Universiteit van Amsterdam)



messages.pdf_cover_qr_code_label

Test Final - Introduction to Python - 22/23

Question 1 – 1_Basics_Logic_3 – 357660.2.5

Suppose you have a variable called **x** which contains an integer between 0 and 10 (both are inclusive).

Which of the following programs will tell you whether **x** contains an odd or even number?

- A**

```
if x in list(range(11))[:2]:  
    print('Even number')  
else:  
    print('Odd number')
```
- B**

```
if x % 2 != 0:  
    print('Odd number')  
else:  
    print('Even number')
```
- C**

```
if x in [1, 3, 5, 7, 9]:  
    print('Odd number')  
else:  
    print('Even number')
```

D All of the programs work as intended.

Question 2 – 1_Basics_Scope_2 – 357609.2.4

Take a look at the following code segment. Will executing this code segment produce an error? If yes, then why?

```
a = 1  
def outer(b):  
    b = 3  
    def inner(c):  
        d = 4  
        return a + c + d  
    return inner(5) * b  
outer(5)
```

- A** This code will produce an error because **a** is a global name that is referenced inside the local scope of **inner()**.
- B** This code will not produce any error and will return 30.
- C** This code will produce an error because **c** is not defined before referencing.
- D** This code will not produce any error and will return 50.

Question 3 – 1_Basics_Variables_2 – 357586.1.3

What will be returned by the following code segment?

```
def main(list_1):
    list_2 = []
    for item in list_1:
        if type(item) == int:
            list_2.append(item)
        else:
            list_2.append(None)
    return list_2

main([0, 1.0, 2, 3, 3.50, 5, 6, 7, 8, '9'])
```

- A** [0, None, 2, 3, None, 5, 6, 7, 8, None]
- B** [0, 1.0, 2, 3, 3.50, 5, 6, 7, 8, 9]
- C** [None, 1.0, None, None, 3.5, None, None, None, None, '9']
- D** [0, None, 2, 3, None, 5, 6, 7, 8, 9]

Question 4 – 2-dictionaries – comprehension – 2 – 359271.1.3

You have to create a dictionary called **roman_nums** where the keys are the integers from 1 to 5 (both are inclusive) and the values are their corresponding Roman numeral symbols.

That is, if we execute the following command:

```
print(roman_nums)
```

the output should be:

```
{1: 'I', 2: 'II', 3: 'III', 4: 'IV', 5: 'V'}
```

Which of the following code segments works as intended?

- A** `roman_nums = {num: roman for num, roman in zip(range(1, 6), ['I', 'II', 'III', 'IV', 'V'])}`
- B** `roman_nums = {roman: num for roman, num in zip(['I', 'II', 'III', 'IV', 'V'], range(1, 6))}`
- C** `roman_nums = {num: ['I', 'II', 'III', 'IV', 'V'][num] for num in range(1, 6)}`
- D** `roman_nums = {num: roman for num, roman in zip(range(1, 6), 'I II III IV V')}`

Question 5 – 2-dictionaries – manipulation – 2 – 358904.1.7

You have two lists representing the names and ages of individuals:

```
names = ['Alice', 'Bob', 'Charlie', 'David']
ages = [25, 30, 35, 40]
```

You need to create a dictionary called **person_dict** where the keys are the names and the values are the ages. However, you want to include only the names that start with the letter 'A' or with the letter 'C'.

So executing the following command:

```
print(person_dict)
should give the following output:
{'Alice': 25, 'Charlie': 35}
```

Which of the following code segments works as intended?

- A**

```
person_dict = {}
for name, age in zip(names, ages):
    if name[0] == 'A' or name[0] == 'C':
        person_dict[name] = age
```
- B**

```
person_dict = {}
for i, name in enumerate(names):
    if name[0] == 'A' or name[0] == 'C':
        person_dict[i] = ages[i]
```
- C**

```
person_dict = {}
for name, age in zip(names, ages):
    if name[0] == 'A' or name[0] == 'C':
        person_dict.append(name:age)
```
- D**

```
person_dict = {}
for i, name in enumerate(names):
    for j, letter in enumerate(name):
        if letter == 'A' or letter == 'C':
            person_dict[i] = ages[j]
```

Question 6 – 2-dictionaries – slicing – 2 – 358889.1.8

You have the following dictionary that represents the performance ratings of employees in a company:

```
employee_ratings = {'John': 8, 'Sarah': 9, 'Michael': 7, 'Emma': 9, 'David': 6}
```

You need to find and print the name of the employee(s) with the highest performance rating.

Which of the following code segments works as intended?

- A**

```
max_rating = max(employee_ratings.values())
for employee, rating in employee_ratings.items():
    if rating == max_rating:
        print(employee)
```
- B**

```
max_rating_employee = max(employee_ratings.keys())
for employee, rating in employee_ratings.items():
    if employee == max_rating_employee:
        print(employee)
```
- C** Both of the given options are correct.
- D** None of the given options are correct.

Question 7 – 2-list – comprehension – 1 – 358870.1.2

What is the output of the following code segment?

```
numbers = [1, 2, 3, 4, 5]
result = [x for x in numbers if x % 2 == 0 and x > 2]
print(result)
```

- A [2, 4]
- B [3, 4, 5]
- ☒ C [4]
- D []

Question 8 – 2-list – manipulation – 1 – 358858.1.4

What is the output of the following code segment?

```
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c']
result = []
for item in zip(list1, list2):
    result.append(item[0] * item[1])
print(result)
```

- ☒ A ['a', 'bb', 'ccc']
- B [1, 2, 3]
- C [1, 'b', 3]
- D ['aa', 'bb', 'cc']

Question 9 – 2-list – slicing – 1 – 358860.2.6

Suppose you have a list called **mylist**. You want to extract every fourth element starting from index 2 (inclusive) to the second-to-last index (inclusive) of that list using the following line of code:

```
subset = mylist[___:___:___]
```

For example, if you have:

```
mylist = [10, 15, 20, 25, 30, 35, 40, 45, 50]
```

then executing the following command:

```
print(subset)
```

should print:

```
[20, 35]
```

What should the blanks be filled with?

- A 2:-1:3
- B 2:len(mylist)-1:3
- ☒ C Both of the given options are correct.
- D None of the given options are correct.

Question 10 – 3_Function_Argument_2 – 357676.2.2

Suppose you have the following function, which takes a list of integers as an argument:

```
def main(x):
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    y = ''
    for index, character in enumerate(alphabet):
        if index in x:
            y += character
    return y
```

What will be returned, if we call the function as follows:

```
main([0, 1, 2, 23, 24, 25])
```

- A** 'abcxyz'
- B** ['a', 'b', 'c', 'x', 'y', 'z']
- C** '012232425'
- D** None of the given options.

Question 11 – 3_Function_Built-in_1 – 357770.1.3

Suppose you have a variable called **x** which contains a string as follows:

```
x = 'Hello world'
```

Which of the following lines of code produce(s) **2** as an output?

- A** `print(x.count('o'))`
- B** `print(x.split(' ')[0].count('l'))`
- C** `print(len(x)//5)`
- D** All of the given options.

Question 12 – 3_Function_Flexible–Argument_2 – 357682.1.10

You need to write a function called **main** which accepts a list of integers as an argument.

The function should return a tuple of two integers, in which the first is the sum of the squares of all even numbers, and the second is the sum of the squares of all odd numbers.

For example, calling the function as:

```
main(1, 2, 3, 4)
```

should return:

```
(20, 10)
```

Which of the following blocks of code fits with the rest of the program and will you give the intended output?

```
def main(*args):  
    total_even = 0  
    total_odd = 0  
  
    # missing lines  
    # ...
```

A

```
for i in args:  
    if i % 2 == 0:  
        total_even += i**2  
    else:  
        total_odd += i**2  
  
return (total_even, total_odd)
```

B

```
for i in args:  
    if i % 2 == 1:  
        total_even += i  
    else:  
        total_odd += i  
  
return (total_even**2, total_odd**2)
```

C

```
for i in args:  
    if i % 2 == 0:  
        total_even.append(i**2)  
    else:  
        total_odd.append(i**2)  
  
return (sum(total_even), sum(total_odd))
```

D

All of the given code blocks work as intended.

Question 13 – 6-string – fstring – 1 – 359329.3.1

You have the following objects loaded in Python:

```
name = "Alice"  
age = 25  
country = "the USA"
```

You need to print the following string:

My name is Alice. I am 25 years old, and I am from the USA.

Which of the following lines of code works as intended?

- A** `message = "My name is {0}. I am {2} years old, and I am from {1}.".format(name, country, age)`
 `print(message)`
- B** `message = f"My name is {name}. I am {age} years old, and I am from {country}."`
 `print(message)`
- C** `message = 'My name is ' + name + '. I am ' + str(age) + ' years old, and I am from ' + str(country)`
 `+ '.'`
 `print(message)`

D All of the given options are correct.

Question 14 – 7-pandas – easy – 2 – 359418.1.5

Suppose you have the following DataFrame called `df`, and assume that pandas is already imported:

	Player	Goals	Assists
0	John	5	2
1	Jane	3	6
2	Alex	2	4

All of the following lines of code make the same selection, but one of the results has a different datatype. Which one?

- A** `df.loc[:, 'Goals']`
- B** `df['Goals']`
- C** `df.loc[[0, 1, 2], 'Goals']`

D `df.loc[:, ['Goals']]`

Question 15 – 8_Datetime_Easy_3 – 358042.3.2

Which code segment gives the same output as the following:

```
date1 = datetime(2023, 6, 1).strftime('%d-%m-%Y')
date2 = datetime(2023, 5, 1).strftime('%d-%m-%Y')
print((datetime.strptime(date1, '%d-%m-%Y') - datetime.strptime(date2, '%d-%m-%Y')).days)
```

You may assume that the datetime and timedelta classes are already imported from the datetime module.

- A** `print((datetime(2023, 5, 1) - datetime(2023, 6, 1)).days)`
- B** `print((datetime(2023, 6, 1) - datetime(2023, 1, 5)).days)`
- C** `print((datetime.strptime('2023-6-1', '%Y-%d-%m') - datetime.strptime('2023-5-1', '%Y-%d-%m')).days)`
- D** `print((datetime.strptime('2023-06-01', '%Y-%m-%d') - datetime.strptime('2023-05-01', '%Y-%m-%d')).days)`

Question 16 – 9_OOP_Easy_2 – 358034.3.1

You have the following definition of a class called **Flight**:

```
class Flight:
    def __init__(self, airline, destination):
        self.airline = airline
        self.destination = destination

        if self.airline == 'KLM' and self.destination == 'Singapore':
            self.duration = 'long'
            self.price = 'expensive'

        if self.airline == 'EasyJet' and self.destination == 'Spain':
            self.duration = 'short'
            self.price = 'cheap'
```

What would be the output if we execute the following statements?

```
print(Flight('KLM', 'Singapore').duration)
print(Flight('EasyJet', 'Spain').price)
```

- A** `'long'`
`'cheap'`
- B** `'short'`
`'expensive'`
- C** `'long'`
`'expensive'`
- D** `'short'`
`'cheap'`

Question 17 – 6-string – operations – 1 – 359326.1.4

Consider the following code snippet:

```
text = "The universe is vast and full of mysteries!"

# Operation 1:
word_list = []
for word in text.split():
    word_list.append(word[::-1])

# Operation 2:
joined_text = ' '.join(word_list, '-')

# Operation 3:
replaced_text = text.replace("mysteries", "secrets")
```

Which operation in the code snippet contains an error?

- A** Operation 1
- B** Operation 2
- C** Operation 3
- D** None of the operations contains an error.

Question 18 – 6-strings – find – 2 – 359535.1.4

Suppose you have the following variables called **sentence1** and **sentence2** which contain strings.

```
sentence1 = "harry, ron, and hermione walked through the halls of hogwarts; their footsteps echoing off the stone walls."
sentence2 = "The marauder's map in harry's hands revealed the secret passageways and hidden rooms of the castle."
```

The sentences contain names, but they are not properly capitalized. You want to create a new string object called **paragraph**, with the proper nouns capitalized in the two sentences and the two sentences connected into one big string.

That is, if we execute the following line of code:

```
print(paragraph)
```

the output should be:

```
"Harry, Ron, and Hermione walked through the halls of Hogwarts; their footsteps echoing off the stone walls.
The Marauder's map in Harry's hands revealed the secret passageways and hidden rooms of the castle."
```

Which of the following code segments achieves what you want?

- A**
- ```
names = ['harry', 'ron', 'hermione', 'hogwarts', 'marauder']
for name in names:
 sentence1[sentence1.find(name)] = name.capitalize()
 sentence2[sentence2.find(name)] = name.capitalize()
paragraph = sentence1 + " " + sentence2
```
- B**
- ```
names = ['harry', 'ron', 'hermione', 'hogwarts', 'marauder']
for word, name in zip(sentence1, names):
    if word == name:
        sentence1 = sentence1.replace(word, name.capitalize())
for word, name in zip(sentence2, names):
    if word == name:
        sentence2 = sentence2.replace(word, name.capitalize())
paragraph = sentence1 + " " + sentence2
```
- C**
- ```
names = ['harry', 'ron', 'hermione', 'hogwarts', 'marauder']
new1 = ' '.join([word.capitalize() if word in names else word for word in sentence1])
new2 = ' '.join([word.capitalize() if word in names else word for word in sentence2])
paragraph = new1 + " " + new2
```
- D**
- ```
for word in ['harry', 'ron', 'hermione', 'hogwarts', 'marauder']:
    sentence1 = sentence1.replace(word, word.capitalize())
    sentence2 = sentence2.replace(word, word.capitalize())
paragraph = sentence1 + " " + sentence2
```

Question 19 – 7-pandas – hard – 2 – 359444.2.8

Consider the following pandas DataFrame called `df`, and assume that Pandas is already imported:

	Name	Age	Occupation
0	John	25	Student
1	Jane	30	Engineer
2	Alex	35	Scientist
3	Lisa	40	Doctor
4	Mark	28	Architect

You need to perform a series of operations on this DataFrame:

- Operation 1: Create a new column called **Name_Length** that contains the length of each person's name
- Operation 2: Replace all occurrences of the occupation "Engineer" with "Software Developer"
- Operation 3: Extract the last two characters of each person's name and store them in a new column called **Name_Suffix**

Which of the following blocks of code performs these operations as intended?

- A**
- ```
df['Name_Length'] = df['Name'].map(lambda x: len(x))
df['Occupation'] = ['Software Developer' if x == 'Engineer' else x for x in df['Occupation']]
df['Name_Suffix'] = df['Name'].map(lambda x: x[-2:])
```
- B**
- ```
df['Name_Length'] = len(df['Name'])
df['Occupation'] = ['Software Developer' if x == 'Engineer' else x for x in df['Occupation']]
df['Name_Suffix'] = df['Name'].map(lambda x: x[-2:])
```
- C**
- ```
df['Name_Length'] = len(df['Name'])
df['Occupation'].map(lambda x: 'Software Developer' if x == 'Engineer' else x)
df['Name_Suffix'] = df['Name'].map(lambda x: x[:-2])
```
- D**
- ```
df['Name_Length'] = df['Name'].map(len)
df['Occupation'] = df['Occupation'].map(lambda x: 'Software Developer' if x == 'Engineer' else x)
df['Name_Suffix'] = df['Name'].map(lambda x: x[-2:-1])
```

Question 20 – 7-pandas – lambda – 2 – 359441.2.1

What will be printed by the following code:

```
import pandas as pd
d = {'Product':['A', 'B', 'C', 'D'], 'Units' :[100, 50, 200, 80], 'Price': [10, 19, 15, 13]}
df = pd.DataFrame(d)
df.loc[:, 'UnitsCorrected'] = df.loc[:, 'Units'].map(lambda x: 0.9*x if x < 100 else x)
df.loc[:, 'Revenue'] = df.loc[:, 'UnitsCorrected'] * df.loc[:, 'Price']
df=df.sort_values(by='Revenue',ascending=False)
df1 = df.loc[:,['Product', 'Units', 'Price']]
print(df1)
```

A

	Product	Units	Price
2	C	200	15
0	A	100	10
3	D	72	13
1	B	45	19

B

	Product	Units	Price
1	B	50	19
0	A	100	10
3	D	80	13
2	C	200	15

C

	Product	Units	Price
2	C	200	15
0	A	100	10
3	D	80	13
1	B	50	19

D

	Product	Units	Price
2	C	200	15
3	D	80	13
0	A	100	10
1	B	50	19

Question 21 – 8_Datetime_Function_2 – 358906.3.10

You have the following definition for a class called **Calendar**:

```
from datetime import datetime, timedelta

class Calendar():
    def __init__(self, event, start, length):
        self.event = event
        self.start = datetime.strptime(start, '%d-%m-%Y %H:%M')
        self.end = self.start + timedelta(minutes=length)
    def check_overlap(self, other):
        return self.start < other.end and self.end > other.start
```

The method **check_overlap()** should return **True** if there are two overlapping events and **False** if the two events are not overlapping.

For example, if we have the following **Calendar** objects:

```
event1 = Calendar('Coffee meeting', '01-06-2023 9:30', 20)
event2 = Calendar('Writing session', '01-06-2023 10:00', 150)
event3 = Calendar('Lunch meeting', '01-06-2023 12:00', 45)
```

the following command should return **False**:

```
print(event1.check_overlap(event2))
```

and the following command returns **True**:

```
print(event2.check_overlap(event3))
```

Does the code segment given above do what you want? If not, then why not?

Hint:

The `timedelta` functions contains the following parameters: days, seconds, microseconds, milliseconds, minutes, hours, and weeks. All the parameters are optional and 0 by default. The `timedelta` object that is created using this function represents a duration.

- ☒ **A** The code segment works as intended.
- ☐ **B** This code segment does not work because the condition provided after the **return** keyword is incorrect.
- ☐ **C** This code segment does not work because the syntax to parse the input string to create a `datetime` object is incorrect.
- ☐ **D** This code segment does not work because the `__init__` method needs 4 parameters and only 3 arguments are passed.

Question 22 – 8_Datetime_Hard_1 – 358890.2.4

You want to create a DataFrame called **df** that has the following format:

	Day	Month	Year
01-01-23	1	1	2023
08-01-23	8	1	2023
15-01-23	15	1	2023
22-01-23	22	1	2023
29-01-23	29	1	2023

Which of the following code segments achieves what you want?

You may assume that the `datetime` and `timedelta` classes are already imported from the `datetime` module, as well as the `pandas` module (via: `import pandas as pd`).

Hint:

The `timedelta` functions contains the following parameters: `days`, `seconds`, `microseconds`, `milliseconds`, `minutes`, `hours`, and `weeks`. All the parameters are optional and 0 by default. The `timedelta` object that is created using this function represents a duration.

- A**
- ```
dates = [datetime(2023, 1, 1) + timedelta(weeks=1*i) for i in range(5)]
data = {'Day' : [date.day for date in dates],
 'Month': [date.month for date in dates],
 'Year' : [date.year for date in dates]}
df = pd.DataFrame(data, index=[date.strftime('%d-%m-%Y') for date in dates])
```
- B**
- ```
dates = [(datetime(2023, 1, 1) + timedelta(days=7*i)).strftime('%d-%m-%Y') for i in range(5)]
data = {'Day' : [date.day for date in dates],
        'Month': [date.month for date in dates],
        'Year' : [date.year for date in dates]}
df = pd.DataFrame(data, index=dates)
```
- C**
- ```
day = pd.Series([(datetime(2023, 1, 1) + timedelta(weeks=1*i)).day for i in range(5)])
month = pd.Series([(datetime(2023, 1, 1) + timedelta(weeks=1*i)).month for i in range(5)])
year = pd.Series([(datetime(2023, 1, 1) + timedelta(weeks=1*i)).year for i in range(5)])
df = pd.DataFrame(data=[day, month, year])
```
- D**
- ```
df = pd.DataFrame(index = [(datetime(2023, 1, 1) + timedelta(weeks=1*i)) for i in range(5)],
                  {'Day' : [date.days for date in index],
                   'Month': [date.months for date in index],
                   'Year' : [date.year for date in index]})
```

Question 23 – 9_OOP_Function_3 – 357922.2.7

You have the following definition of a class called **Pack**:

```
class Pack:
    def __init__(self, size, value = 0):
        self.size = size
        self.value = value

    # missing lines
    # ...
```

You want to add to this class a method called **fitting()** that calculates how many times another object of the class **Pack** would fit inside this object.

The method should return the number of times these other objects would fit completely in this object, and the final value. The final value is the sum of all values of all objects that fit in it, plus the value of the carrying object **self**.

For example, if we execute the lines of code:

```
backpack = Pack(size=15, value = 20)
book      = Pack(size=4, value=10)
backpack.fitting(book)
```

the output should be:

Amount: 3. Value: 50.

Which of the following code segments should you place on the missing lines in the class definition to achieve what you want?

A

```
def fitting(self, other):
    if self.size > other.size:
        self.total = (self.size // other.size) * other.value + self.value
        return f'Amount: {self.size // other.size}. Value: {self.total}.'
    else:
        return f'Amount: 0. Value: {self.value}.'
```

B

```
def fitting(self, other):
    if self.size > other.size:
        return f'Amount: {self.size // other.size}. Value: {other.value * other.size}.'
    else:
        return f'Amount: 0. Value: {self.value}.'
```

C

```
def fitting(self, other):
    self.amount = self.size / other.size
    self.total = self.amount * other.value
    return f'Amount: {self.amount}. Value: {self.total}.'
```

D

```
def fitting(self, other):
    self.amount = self.size / other.size
    self.total = self.amount * other.value
    return 'Amount: {}. Value: {}'.format(self.amount, self.value if self.size > other.size
else self.total)
```


Question 24 – 9_OOP_Hard_1 – 357979.3.2

You have the following definition for a class called Piece

```
class Piece:
    def __init__(self, diagonal, straight, initial_position):
        self.diagonal = diagonal
        self.straight = straight
        self.initial_position = initial_position

    def check_move(self, target_position):
        self.target_position = target_position
        self.row_diff = self.target_position[0] - self.initial_position[0]
        if abs(self.row_diff) > 3:
            return False
        self.col_diff = self.target_position[1] - self.initial_position[1]
        if abs(self.col_diff) > 3:
            return False

        if self.diagonal and (not self.straight):
            return abs(self.row_diff) == abs(self.col_diff)
        if self.straight and (not self.diagonal):
            return abs(self.row_diff) == 0 or abs(self.col_diff) == 0
        if self.straight and self.diagonal:
            return (abs(self.row_diff) == abs(self.col_diff)) or (abs(self.row_diff) == 0 or
abs(self.col_diff) == 0)
```

And the creation of three objects with the help of the class definition:

```
a = Piece(diagonal=True, straight=False, initial_position=(0, 0))
b = Piece(diagonal=False, straight=True, initial_position=(4, 3))
c = Piece(diagonal=True, straight=True, initial_position=(9, 6))
```

If we run the following statements:

```
print(a.check_move((4, 4)))
print(b.check_move((5, 5)))
print(c.check_move((6, 6)))
```

This will print 3 Boolean values. Of these values, how many times will this be True?

- A 0
- ☒ B 1
- C 2
- D 3